JSON Refiner Advanced Edition

By

Asina J (230833018020112009)
Aswini M (230833018020112010)
Blessy D G (230833018020112011)
Brintha K P (230833018020112012)

of

ARUNACHALA ARTS AND SCIENCE (WOMEN) COLLEGE

Submitted to the

In partial fulfilment of the requirement

for the award of the degree of

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

MANONMANIAM SUNDARANAR UNIVERSITY
TIRUNELVELI – 627012
MARCH -2026

JSON Refiner Advanced Edition

Professional JSON Processing Tool with Gradio Interface

Project Description

JSON Refiner – Advanced Edition

JSON Refiner Advanced Edition is a powerful tool created using Python for efficient JSON data processing. The system helps users parse, validate, organize, and transform JSON data through an interactive interface developed with **Gradio**. It simplifies the process of converting raw or unstructured data into properly formatted JSON structures.

The platform mainly focuses on improving the quality and consistency of JSON data. It supports intelligent JSON generation, automatic identification of correct data types, schema validation, case format conversion, and restructuring of nested JSON objects. These capabilities help ensure that the final output follows proper JSON standards and is suitable for real-world applications.

The system uses advanced parsing techniques along with Python's flexible data handling features to analyze key-value inputs. Based on the provided data, it automatically determines suitable data types such as strings, numbers, booleans, null values, arrays, and objects. It also standardizes key naming formats including **snake_case**, **camelCase**, **kebab-case**, and **PascalCase**, which improves readability and maintains consistency across datasets.

In addition, the platform provides schema validation features that allow users to check whether the JSON data structure follows defined rules. Various transformation tools are also available to flatten complex nested structures or rebuild them into hierarchical formats whenever required.

With its clean and interactive interface powered by Gradio, JSON Refiner Advanced Edition makes JSON handling easier for developers, data analysts, and students. The system provides clear validation feedback, multiple formatting options, and reliable transformation capabilities to ensure high-quality JSON outputs.

Overall, JSON Refiner Advanced Edition offers an intelligent and user-friendly approach to JSON processing by combining advanced data analysis methods, flexible transformation tools, and a modern interface that improves productivity and data reliability.

scenarios

Scenario 1: Converting CSV Data into JSON

A data analyst needs to transform information stored in a CSV file into a structured JSON format. The CSV contains simple key-value pairs such as *name: John, age: 28, active: true, score: 95.5*. The system reads this input, automatically determines the appropriate data types, and converts the information into well-structured JSON. It also standardizes key names using consistent naming styles and produces clean output suitable for production systems. Before importing the data into a database, the analyst can also verify it using a JSON schema to ensure accuracy and reliability.

Scenario 2: Standardizing API Responses

A backend developer works with several external APIs that return JSON data with inconsistent naming formats and structures. The system processes these responses and converts all keys into a unified format, ensuring consistent naming conventions across the dataset. It can combine responses from multiple APIs into a single organized structure, remove unnecessary null or empty values, and check the data against a predefined schema. This allows the developer to deliver clean and consistent data to the frontend without writing additional transformation logic.

Scenario 3: Managing Configuration Files

A DevOps engineer maintains application configuration settings in a simple text format and needs them converted into JSON for deployment automation. The system reads configuration entries such as *database_host: localhost, port: 5432, ssl_enabled: true,* and *retry_count: 3*, automatically identifying the correct data types. It also checks for missing required fields and prevents duplicate keys while providing clear error messages. Finally, it produces valid JSON files that can be directly used in infrastructure-as-code or automated deployment tools.

Scenario 4: Flattening Nested JSON for Data Analysis

An analytics team receives application data in deeply nested JSON structures that are difficult to analyze using spreadsheet tools. The system simplifies these structures by converting nested objects into a flattened format using dot notation. For example, a structure like *user → profile → age* becomes *user.profile.age*. This format allows the data to be easily exported and analyzed in tools such as spreadsheets. When required, the system can reconstruct the original nested JSON structure, ensuring the transformation process remains fully reversible.

Scenario 5: Validating Data with JSON Schema

A quality assurance team needs to verify JSON data according to predefined organizational standards. The system processes the raw input, applies necessary transformations, and validates the data using JSON Schema rules. It identifies issues such as missing fields, incorrect data types, or invalid structures and provides detailed feedback for correction. By performing these checks early in the workflow, the system helps maintain consistent and reliable data across different platforms and applications.

Architecture Overview

JSON Refiner Advanced is designed as a modular system that combines a Python-based backend with advanced JSON processing techniques to perform intelligent data transformation. The architecture focuses on maintaining data accuracy, improving user interaction, and generating production-ready results. An interactive interface built using Gradio enables users to easily perform JSON parsing, validation, and transformation while receiving clear feedback and error messages.

Core Technologies

The platform uses several modern technologies to ensure reliable performance and efficient data processing:

- Python 3.8+ – Main programming language used to implement JSON processing logic and system functionality.
- Gradio 4.0+ – Provides an interactive web interface that allows users to interact with the application easily.
- JSON5 – Enables flexible JSON parsing and supports extended JSON syntax.
- JSON Schema – Used to validate JSON structures and ensure that data follows predefined rules.
- Regular Expressions – Helps in pattern matching and advanced text parsing during data processing.
- Google Colab – Cloud-based environment used for development, execution, and sharing of the application.
- HTML5 / CSS3 – Used for modern web-based interface design and presentation.

Pre-requisites

Software Requirements

The following software tools and resources are required to run the system:

- Python 3.8 or higher – Available from the official Python website.

- Google Account – Needed to access and run notebooks in Google Colab.

- pip – Python package manager used to install required libraries.

- Web Browser – Any modern browser such as Google Chrome, Mozilla Firefox, Safari, or Microsoft Edge.

- Internet Connection – Required for accessing Colab and enabling Gradio interface sharing.

Knowledge Prerequisites

To effectively work with the system, users should have basic knowledge of the following topics:

- Python Programming Basics – Understanding of functions, dictionaries, lists, and string operations.

- JSON Concepts – Knowledge of JSON structure, data types, and validation methods.

- Regular Expressions – Basic understanding of pattern matching used for parsing text data.

- Gradio Framework – Familiarity with UI components and callback functions.

- Data Validation Techniques – Awareness of schema validation using JSON Schema.

- Google Colab Environment – Basic usage of notebooks and code execution in cells.

Hardware Requirements

The system does not require high-end hardware and can run on standard computers:

- Processor: Any modern CPU such as Intel i3 or an equivalent processor.

- RAM: Minimum 2 GB (cloud resources are provided when using Google

Colab).

- Storage: At least 500 MB of free space for application files and generated outputs.

- Display: Minimum resolution of 1024 × 768 to ensure proper interface visibility.

Project workflow

The development of JSON Refiner Advanced follows a structured workflow consisting of multiple phases. Each phase focuses on building and integrating specific components of the system, starting from environment configuration to final testing and deployment.

Phase 1: Environment Setup and System Architecture

Activity 1.1: Dependency Installation and Environment Configuration

- Prepare a **requirements.txt** file containing all required libraries.

- Configure the development environment using **Google Colab**.

- Install necessary Python packages such as **Gradio**, **JSON5**, **jsonschema**, **python-dotenv**, and **Pydantic**.

- Confirm that all dependencies are installed and functioning properly.

Activity 1.2: Core Data Structure Design

- Define a **CaseStyle enumeration** to support different naming conventions.

- Develop the **JSONRefinerCore** class to manage all processing operations

- Create a type detection mechanism to automatically determine appropriate data types.

- Integrate logging and error tracking features to monitor system behavior.

Phase 2: JSON Processing Engine Development

Activity 2.1: Automatic Type Detection

- Develop an **infer_type()** function to detect and assign suitable data types.
- Handle boolean values such as *true/false, yes/no,* and *on/off*.
- Manage null-like values including *null, none, nil,* and *n/a*.
- Implement number recognition for both integers and floating-point values.
- Enable parsing of arrays and objects from JSON-formatted strings.
- If a value cannot be identified, treat it as a string by default.

Activity 2.2: Key Formatting and Case Conversion

- Create methods to convert key names into different formats:

  - **snake_case**
  - **camelCase**
  - **kebab-case**
  - **PascalCase**

- Implement a **normalize_key()** function that applies the selected case format automatically.

- Perform tests to ensure that each naming style works correctly.

Phase 3: Data Validation and Transformation

Activity 3.1: JSON Schema Validation

- Develop a **validate_json_schema()** function.

- Utilize the **JSON Schema** standard with the **jsonschema** library for validation.

- Format validation errors clearly for user understanding.

- Test the validation process with different schema definitions.

Activity 3.2: Required Field Verification

- Implement a **check_required_fields()** function to verify mandatory fields.

- Ensure that all essential attributes are present in the data.

- Generate reports listing any missing fields.

- Combine these results with schema validation outputs.

Activity 3.3: JSON Data Transformation

- Create **flatten_json()** to convert nested JSON structures into flat key-value format.

- Implement **unflatten_json()** to rebuild the original nested structure.

- Develop **merge_json_objects()** for combining multiple JSON datasets.

- Add **remove_null_values()** to eliminate unnecessary empty values.
- Test these transformations using complex datasets.

Phase 4: User Interface Development

Activity 4.1: Main Refining Interface Design

- Build input fields where users can enter key-value text data.
- Add radio buttons to allow users to choose the preferred case format.
- Provide options such as removing null values or flattening JSON structures.
- Display the generated JSON output in a formatted code panel.
- Include status indicators and statistics panels to show processing results.

The interface is implemented using **Gradio**, which enables interactive web-based user interaction.

Phase 5: Integration, Testing, and Deployment

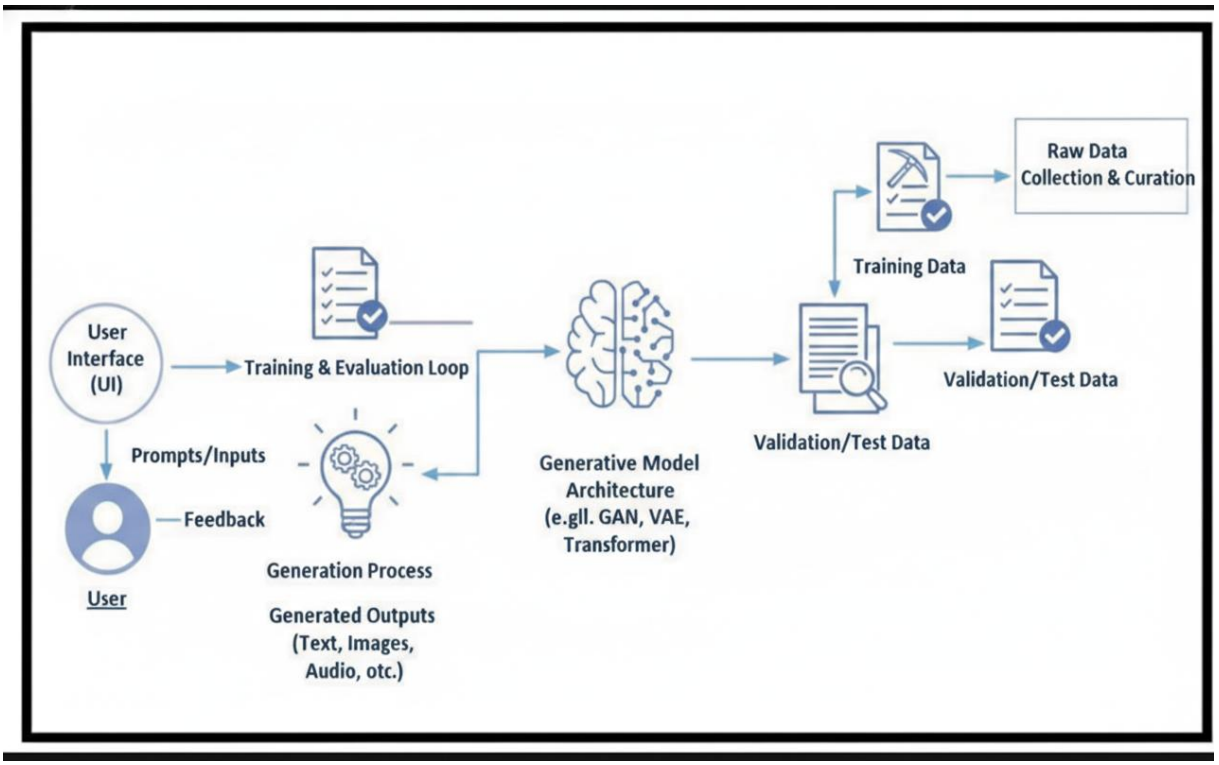
Activity 5.1: Deployment on Google Colab

- Integrate all interface components and backend logic.
- Ensure proper data communication between modules.
- Verify the complete transformation workflow.
- Check error-handling mechanisms for reliability.
- The system is executed and shared through **Google Colab**.

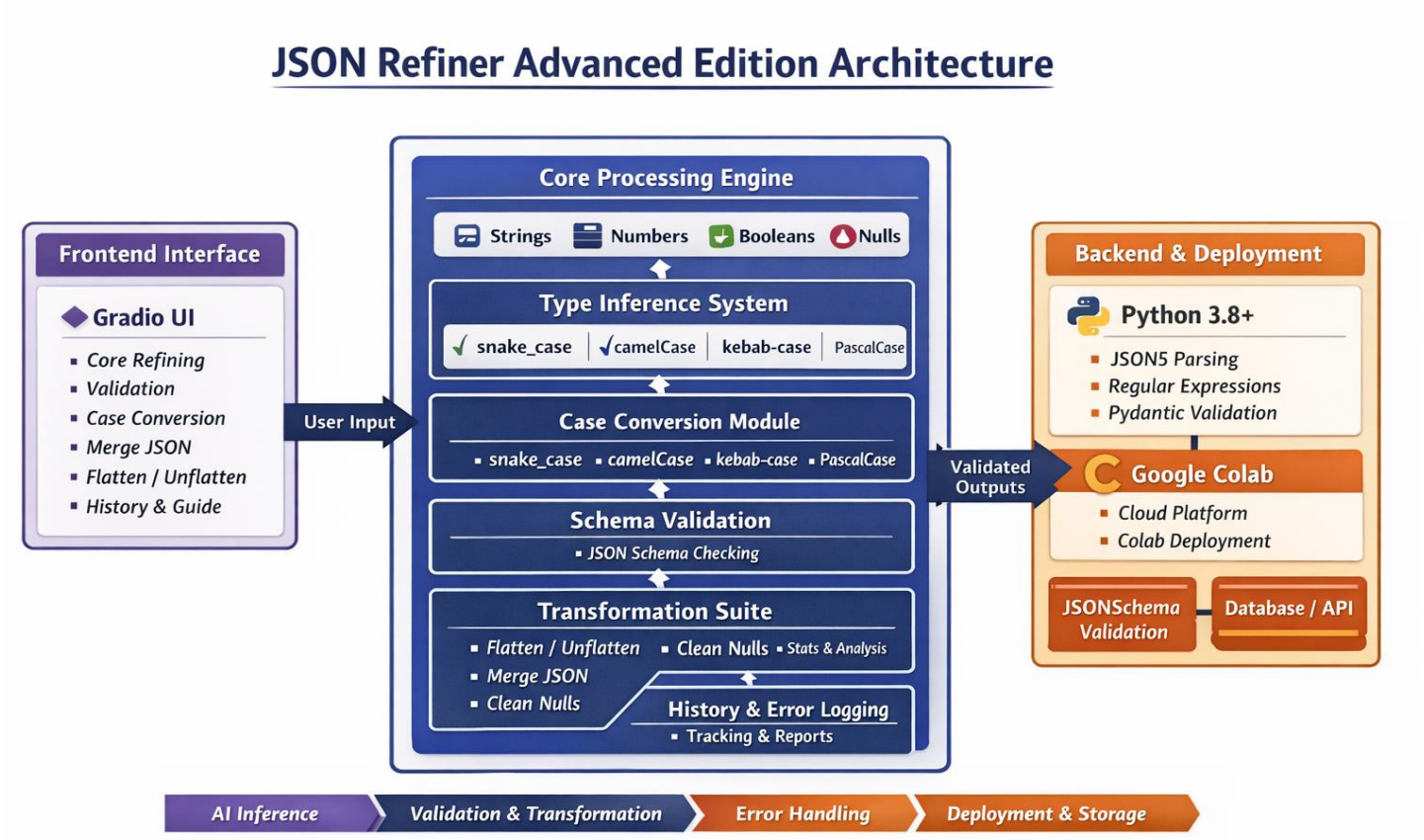
Activity 5.2: System Testing

- Evaluate each case conversion format for accuracy.
- Verify the correctness of automatic type detection.
- Test schema validation using multiple JSON schemas.
- Check merging functionality when conflicting keys appear.
- Assess flattening and unflattening with deeply nested JSON data.

Technical Architecture



Project Structure



Conclusion

JSON Refiner Advanced Edition provides an effective solution for intelligent JSON data processing by combining powerful Python-based data manipulation with an easy-to-use interactive interface. Using frameworks such as **Gradio** and the cloud-based environment of **Google Colab**, the platform enables users to perform JSON parsing, validation, and transformation quickly and efficiently. This makes the system accessible not only to developers but also to data analysts and users with minimal technical background.

The project demonstrates how automated tools can simplify complex data processing tasks while maintaining high standards of accuracy and reliability. Features such as automatic data type detection, multiple key naming format conversions, advanced JSON transformation methods, and strong validation mechanisms help ensure that the generated output is clean, consistent, and suitable for real-world applications. Additionally, the system keeps track of operations and provides clear feedback, allowing users to manage JSON data more effectively.

Future Enhancements

Due to its modular design, the platform can be easily expanded with new capabilities in future versions. Potential improvements include the addition of customizable transformation rules, regular expression–based data extraction, and integration with popular Python data analysis libraries such as **Pandas** and **NumPy**. Other possible enhancements include support for batch processing, automatic API generation, user-defined validation rules, and template-driven JSON transformations.

Further development may also introduce additional case formatting styles, machine learning–based type detection, and data anonymization features for handling sensitive information. Integration with cloud storage platforms could enable large-scale data processing and improved collaboration.

Overall, with continuous improvements, JSON Refiner Advanced Edition has the potential to grow into a comprehensive data processing platform capable of handling multiple data formats while maintaining its core focus on usability, accuracy, and efficient data management.